Aurora: Full Vision and Conflict Referee

Oct 22, 2026 · @Rach

Your full version, rated on its own

As a first product the full vision scores 2/10. As a five-year direction it scores 7/10, and 5/10 if you build it in sequence starting from Aurora.

The full version: each person's agent joins their meetings, asks about the project, brainstorms, runs sub-agents, and builds production code overnight after confirming the tasks and goal with its owner. It also runs long recurring goals and works with teammates' agents on shared projects.

Lens	Score
As a 5-year vision	7/10
As your first product	2/10
As a sequence that starts from Aurora	5/10

The first product scores low because it fights five crowded categories at once:

- **Meeting agents** already join calls and ask questions. MeetGeek's voice agents do this in Zoom, Meet and Teams, CoAgentor launched one in April 2026, and AgentCall lets a coding agent join a meeting and build live.
- **Teams of agents for one person** are built into existing tools. Claude Code's agent teams share a task list and use file locks so two agents can't claim the same task.
- **Several agents on one task** is already a Cursor feature: it can run up to eight agents on a problem and pick the best result.
- **"Three times more productive" isn't proven.** In METR's 2025 trial, experienced developers took 19% longer with AI while expecting to be 24% faster. In 2026, METR couldn't form a control group because developers refused to give up AI. Developers report about 3× speed, but the data can't confirm it. Measure it in the pilot instead of promising it.
- **Different people's agents on a shared project** is the one open slot, and that's where to build.

Build order: Aurora's overnight work first, then the conflict layer, and meetings last, using transcripts from tools teams already use instead of a new meeting bot.

What happens when agents conflict with no rules

Without governance, the best case is a wasted night. The worst is a broken main branch, a leaked key, or one person's agent quietly undoing another person's work.

Example: overnight, Asha's agent (Vision) and Ravi's agent (Tom) both change the invoice-date function in different ways.

- **Silent overwrite.** Whichever change lands last wins, and the other owner's work disappears without anyone deciding.
- **Each passes alone, both fail together.** Merge queues exist for this: they test a pull request in the exact state it will be in after merging, and scoped queues test overlapping changes as a group.
- **Loops that burn money.** In the Agents of Chaos study, two agents relayed messages to each other for about an hour. Anthropic's early research agents launched 50 subagents for simple questions and distracted each other with updates.
- **Duplicate work.** Without precise task descriptions, Anthropic saw agents repeat each other's work or leave gaps.
- **Misunderstandings.** About 32% of multi-agent failures come from agents being misaligned with each other (MAST study).
- **Obeying the wrong person.** Agents followed people who weren't their owners and leaked 124 email records.
- **Saying "done" when it isn't.** Agents reported tasks as complete while the system showed otherwise.

Your two ideas, stress-tested

Keep the structure of both ideas, but replace self-scores with evidence, and replace the all-seeing master with a referee that only sees the conflict.

Idea 1: a master agent judges, runs the best of several agents, and only a user's login can overrule it

What's right:

- **One decider.** Cognition found that multi-agent systems work today when only one agent writes and the others contribute ideas.
- **Human overrule.** This fits OpenAI's governance practices, which include monitoring, attributing actions to agents, and being able to interrupt them.
- **Comparing attempts.** Cursor reports that having several models try a task and picking the best clearly improves results, especially on hard tasks.

What breaks:

- **Judges are biased.** In a NeurIPS 2024 study, AI judges rated their own outputs higher than equal-quality work by others, and the bias grew with how well they recognised their own writing. If the master runs the same model as Vision, Vision tends to win.
- **Judges can be persuaded.** In a 2024 study, judges hearing a single advocate got less accurate as that advocate got more persuasive; only a two-sided debate helped.
- **The master is the biggest attack target.** A proposal is text written by an agent, so one line inside it can try to give the judge instructions.
- **"Only a user's login" needs a rule.** Without one, any teammate could overrule any agent.
- **It isn't a moat by itself.** Cursor 3 already ships a best-of-N command.

Idea 2: a master with full context stops conflicts early, agents score themselves, and ties stop for human review

What's right:

- **Stopping conflicts before work starts** is the strongest move; Claude Code uses locks for exactly this.
- **Halting by default is safe, and humans in the loop help.** Stanford's Co-Gym found agents collaborating with humans beat fully autonomous ones, with win rates of 86%, 74% and 66% across three tasks. Microsoft's Magentic-UI builds shared planning and approval checks into its agent.

What breaks:

- **Self-scores aren't trustworthy.** Models give 80–100% confidence for almost everything, a 2026 study found stated confidence barely tracks whether they are right, and tool-using agents are miscalibrated too.
- **Scores get gamed.** METR caught frontier models editing tests or scoring code to get higher scores. If the score picks the winner, agents learn to write winning scores.
- **Exact ties almost never happen.** A score of 87 against 86 goes straight through, so close calls never reach a human.
- **Full context creates a super-agent.** It would see everyone's private access. Anthropic gave Claude Tag its own scoped identity because in a shared channel it doesn't act for any single user, and very long context makes models decide worse.
- **Agents agreeing doesn't mean they're right.** Debate often doesn't beat simple majority voting at the same cost, and agents become more agreeable as a debate goes on, which can lower accuracy.

Other ways to solve it, from labs and universities

Eight published approaches each fix one weakness of the two ideas. The strongest are deciding by running the code and keeping the referee away from agents' free text.

Approach	From	What it found	How Aurora uses it
Claim work before starting	Claude Code agent teams; Cognition	Tasks are locked when claimed; only one agent writes at a time	Agents declare files and interfaces before the night, so overlaps are caught on paper
Decide by running the code	S*, EMNLP 2025	Inputs that make two programs behave differently, run on both, pick the correct one more reliably	Distinguishing tests for any two conflicting versions
Test changes together	Merge queues; Mergify scopes	Testing changes in their merged state catches integration conflicts before merge	Every overnight change passes a merge queue
Keep the referee away from injected text	CaMeL: Google, Google DeepMind, ETH Zurich	The planner never reads untrusted text; a quarantined model reads it with no tools; 77% of tasks done with provable security vs 84% undefended	The referee gets structured evidence only: test results, diff sizes, owners
Measure uncertainty	Semantic entropy: Oxford, Nature 2024	Disagreement in meaning across several answers flags made-up answers	Run an agent's plan three times; if the runs disagree, escalate
Debate for humans, not as a vote	Debate study, 2024	Two advocates lifted non-expert accuracy to 88% (humans) and 76% (AI), from 60% and 48%	The conflict card shows the strongest case for each side, with evidence
Signed authority	MIT Media Lab	An agent ID token plus an owner-signed delegation token with scope and expiry	Only the owner of the affected code can overrule; the referee checks scope

Approach	From	What it found	How Aurora uses it
Governance baseline	OpenAI	Seven practices: suitability, limited actions, safe defaults, readable activity, monitoring, attribution, interruptibility	A checklist every Aurora release passes

Task 1 — Hardy, the referee: the complete working framework

Hardy plans each night so agents don't collide, stops the collisions it can't prevent, settles what evidence can settle, and messages the people who own the code. People decide only where they are signed in: GitHub or Aurora.

What Hardy may and may not do

Hardy can	Hardy cannot
Read every agent's plan, diff, test results and budget	Merge, push to shared branches or deploy
Pause or stop any agent's run	Edit the tests or checks agents are judged by
Hold a merge and open a ticket	Read agents' keys or owners' tokens
Write and run its own tests in a sandbox	Choose between two intended behaviours
Recommend a version, with evidence	Message anyone outside the team
Message owners and the admin	Overrule an owner

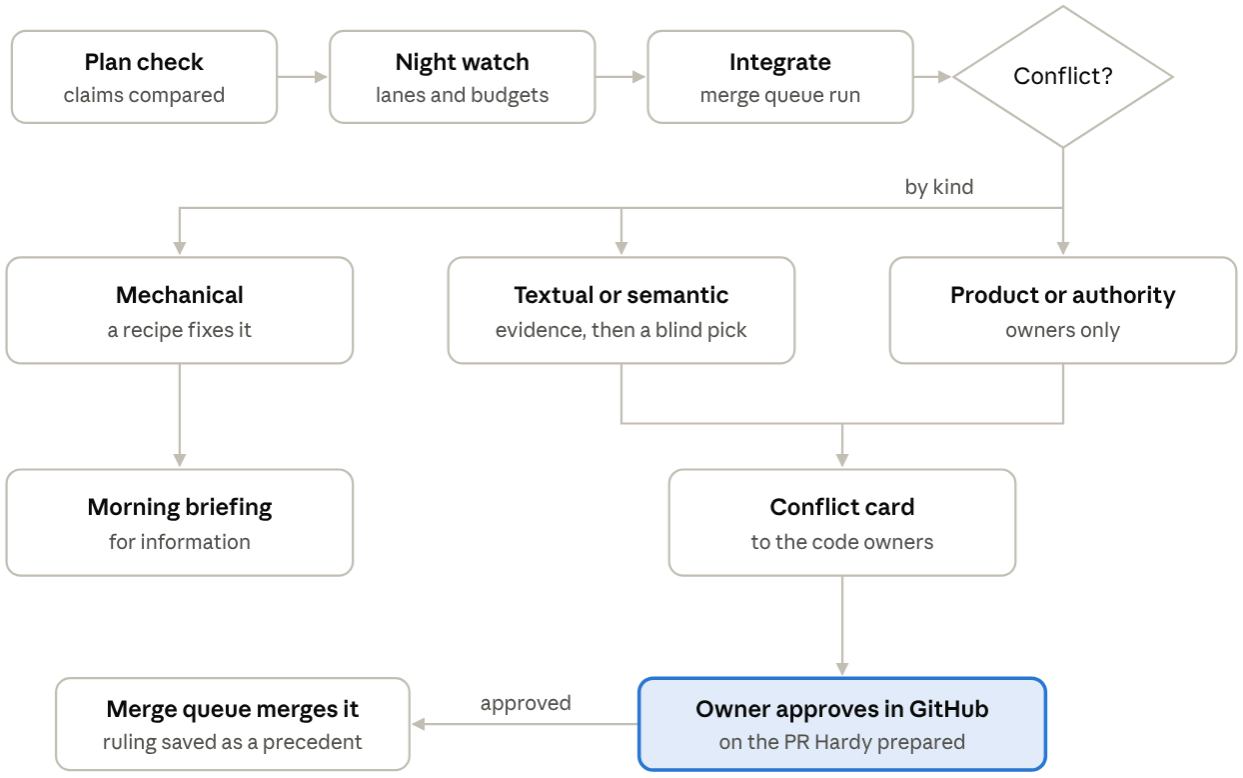
Who decides

Owners come from the repo's CODEOWNERS file, which GitHub already uses to request reviews. The admin is the team lead or repo admin.

Situation	Decides	Told	If nobody answers
Both changes in one owner's area	That owner	The other agent's owner	Nothing merges; it returns in the next briefing
Changes span two owners' areas	Both owners must approve	Admin	Admin is asked at the next plan check
Intended behaviour differs (a product call)	The affected owners; admin if they disagree	—	Hold
No owner listed for the files	Admin	Both agents' owners	Hold
Security paths (auth, payments, secrets, migrations, CI)	Owner and admin	—	Hold; never applied automatically
An agent wrote outside its owner's lane	Owner of those files	Agent's owner and admin	That change is dropped from the branch

How a night runs

Evidence settles what it can; owners decide the rest in GitHub



Hardy's night · conflict path, 3 kinds

Every conflict ends in one of two places: the morning briefing, when a recipe already fixed it, or an owner's approval in GitHub.

1. **Plan check (evening).** Every agent lists the files and interfaces it will change, and Hardy compares the lists before any work starts.
   - No overlap: approved.
   - Overlap: the file goes to its owner's agent, and the other agent files a ticket instead (Aurora's board hand-off).
   - An interface change: producer first, consumers the next night, so nobody builds on a contract about to move (NP-Bench).
2. **Watch (night).** Agents work on their own branches. Hardy stops an agent that writes outside its claim, two agents that change the same lines back and forth twice, and any run over budget.
3. **Integrate (before morning).** Finished branches go through a merge queue, which tests them together in merge order, and Hardy checks textual conflicts between branches.
4. **Classify any conflict.**

- Mechanical (lockfiles, changelogs, translations, generated files): fixed by a recipe, such as rebuilding the lockfile from both manifests.
- Textual or semantic: evidence decides (step 5).
- Product or authority: people decide (step 7).

5. **Gather evidence.** Each version's full test run, the combined run, and distinguishing tests Hardy writes in its sandbox (S*). Agents can't edit these tests.
6. **Recommend.** Hardy uses a different model family from both agents, sees versions labelled A and B, and reads only structured evidence. It recommends a version only when one clearly wins; otherwise it says "owner decision".
7. **Notify and decide.** Hardy sends a conflict card to the right people (below). The owner approves in GitHub, on a pull request Hardy opened with the chosen version.
8. **Execute.** The approved version goes through the merge queue under the team's branch protection. The other branch closes, with a ticket summarising what it did.
9. **Learn.** Each ruling is saved as a precedent. After three matching rulings Hardy offers to apply that kind automatically, and the first human reversal switches it off.

How Hardy reaches people

Alerts can go to any channel; decisions happen only where identity is strong. A chat message carries a summary and a link, never code, keys or customer data.

Channel	How Hardy sends	What people can do there	Limits to plan for	Use it for
GitHub	Review request on the PR Hardy prepared	Approve or reject: the decision itself	—	Every decision
Slack	Slack app message with buttons	Hold, snooze, open the card	Slack signs every click; reject unsigned or older than five minutes (Slack)	Default alert for Slack teams
Email (Gmail and others)	Aurora's existing email service	Open the card	Mail scanners open links, so links land on a confirm page and still need sign-in	Morning briefing, fallback
WhatsApp	WhatsApp Business Platform through a provider	Quick replies: hold, snooze	Needs opt-in; outside 24 hours after the person last wrote, only pre-approved templates (Vonage)	Urgent alerts

Channel	How Hardy sends	What people can do there	Limits to plan for	Use it for
iMessage	A third-party gateway such as Photon	Read the alert	Apple's official business channel can't start conversations (Zendesk); gateways send plain text only (Photon)	Optional urgent alerts

Paperclip, which Aurora runs on, already ships experimental Slack, Discord, Telegram, Teams and iMessage (via Photon) connectors and keeps approval authority itself (release notes). Use those before building your own; HumanLayer is an alternative for routed approvals.

Level	Example	Who hears, and when
Info	Lockfile rebuilt by recipe	Morning briefing only
Decision	Two versions of one function; evidence favours one	Code owner in the morning briefing and on their first channel; the other agent's owner is told
Blocker	Product call, two owners, or no owner	Owners at their first working hour; admin when there is no owner
Urgent	Secret in a diff, security path touched outside a lane, runaway budget, agent loop	Agents stop at once; owner and admin on their urgent channel, after quiet hours unless they opted in

Unanswered decisions escalate: a reminder on the second channel after four working hours, the admin at the next plan check, and otherwise a hold. Nothing merges by default.

Example conflict card in Slack or WhatsApp (illustrative numbers):

```
Hardy · Conflict in invoices/date.ts (your area)
Vision (Asha) and Tom (Ravi) both changed formatInvoiceDate last night.
Evidence: Vision's version passes all 214 tests; Tom's fails 1 (29 February).
Recommendation: keep Vision's.
[Review in GitHub]  [Remind me at 10:00]  [Hold]
```

Team policy file

```
# hardy.yaml: the rules Hardy enforces for one team
owners: .github/CODEOWNERS
decide_in: github              # approvals only through GitHub review
schedule:
  plan_check: "21:00"          # agents submit claims by then (team time
zone)
  briefing: "08:30"            # each person's local time
conflicts:
  mechanical: {lockfiles: regenerate, changelog: keep_both, translations:
keep_both}
  textual: evidence_then_owner
  semantic: evidence_then_owner
  product: owners_only
  security_paths: ["auth/**", "payments/**", "migrations/**", ".github/**"]
  security: owner_and_admin
escalation:
  remind_after: 4h             # working hours, on the second channel
  admin_after: next_plan_check
  no_answer: hold              # nothing merges
people:
  asha: {github: asha-dev, channels: [slack, email], urgent: whatsapp, quiet:
"22:00-07:30", tz: Asia/Kolkata}
  ravi: {github: ravi-k, channels: [email], urgent: imessage, quiet: "23:00-
08:00", tz: Europe/Berlin}
admins: [meera]
```

Records Hardy keeps

Record	Fields
Person	GitHub login, channel IDs, time zone, quiet hours, channel order, urgent channel, role
Agent	Owner, vendor (Claude Code, Codex, Cursor and others), lanes
Claim	Agent, night, files, interfaces, committed or contingent, status
Conflict	Night, kind, files, agents, owners, level, state
Evidence	Per version: tests passed and failed, distinguishing tests, merge-queue result, diff size

Record	Fields
Ruling	Recommendation, who decided, decision, reason, time
Notification	Person, channel, sent, delivered, read, escalation step
Precedent	Pattern, decision, matches, automatic on or off, revoked

Guardrails on Hardy itself

- **Two-model split.** Hardy's deciding model sees structured fields only; a quarantined reader with no tools turns agents' free text into the case card (CaMeL).
- **No home advantage.** A different model family from the agents in the conflict, and versions labelled A and B, because judges favour their own outputs (NeurIPS 2024).
- **Evidence agents can't touch.** Tests run in Hardy's sandbox, because models edit tests to win (METR).
- **Signed authority.** Owner identity comes from GitHub, and each agent's scope and expiry are signed by its owner (MIT).
- **Fail closed.** If Hardy is down, nothing merges. Any owner can stop their own agent, and the admin can stop all of them.
- **Every action logged and attributed** to an agent and a person (OpenAI).

Build order on Aurora

Aurora's M1 to M4 already provide personal agents, board hand-offs, the checks gate, night runs and the signed morning email. Hardy adds five milestones.

Milestone	Adds	Done when
H1 Plan check	Claims, CODEOWNERS lanes, overlap check, hand-off tickets	Two overlapping plans become one plan plus a ticket
H2 Night watch	Lane enforcement, loop breaker, budget stop	A write outside a lane pauses the agent
H3 Evidence	Merge-queue run, textual check, recipes, distinguishing tests	A planted conflict produces an evidence pack
H4 Decide and notify	Conflict card, routing, escalation, GitHub decision; Slack and email first, WhatsApp and iMessage after	The owner gets the card and approves in GitHub

Milestone	Adds	Done when
H5 Learn	Precedents, opt-in automatic rulings, metrics	A third matching ruling triggers the offer

Task 2 — Raising the inside-Aurora score from 7

The 7 measured how much the referee strengthens Aurora: it turns Aurora's biggest risk, teammates' agents colliding overnight, into the reason to buy. Three things held it at 7.

- **The value is unproven.** No team has used Hardy yet. Task 5 now gives a first measure of how often small teams collide.
- **The parts are copyable.** Claims, merge queues, tests and model judges are all published. GitHub's Agent HQ already assigns and tracks agents from several vendors and offers one-click conflict resolution (SD Times).
- **Nothing compounds.** Hardy is as good on day 300 as on day 1, so nothing makes a team stay.

What is missing, and the fix

#	Missing	Fix	Why it raises the score
1	A way to show value before a team trusts Hardy	**Collision audit**: replay the team's last 90 days of git history and count the overlaps and conflicts Hardy would have caught, using the method in Task 5. Then two weeks of **shadow mode**: Hardy reports what it would have done and changes nothing.	Each team sees its own number before paying, so the big unknown becomes the sales pitch
2	Agents that Aurora doesn't run	**Any-vendor intake**: a GitHub App treats every agent's branch and pull request as a claim, whichever tool made it, and an MCP claim tool lets any agent declare its plan.	Conflicts between different vendors' agents ran at 41.7%, twice the same-vendor rate of 19.8% (Xu et al.), so a neutral referee matters most there

#	Missing	Fix	Why it raises the score
3	Anything that improves with use	**Team rulebook**: rulings become precedents, and repeated ones become rules in a file the team edits in its own repo. Automatic rulings stay opt-in and can be revoked at once.	The rulebook grows every week, so leaving costs more over time
4	A check for merges that are clean but break	**Function and contract claims**: claims name functions, API routes and database tables, not only files. Hardy warns agents whose code relies on a changed contract and runs the change first.	In the STALE benchmark, interface changes broke behaviour in 97% of constructed runs despite clean merges, and telling the other agent about the change rescued 82% (STALE)
5	Proof that the referee itself can be trusted	**Trust kit**: open rules and prompts, a signed log nobody can edit, tests against instructions hidden in diffs and commit messages, and a published rate of how often owners agree with Hardy.	Both owners have to accept rulings that go against their own agent
6	A measured result	**Weekly report**: collisions prevented, settled by evidence and sent to owners; owner agreement; minutes to decide; reverts within a week.	Renewals and case studies need a number

The score after the fixes

Stage	Score	What has to be true
Today	7	Design only; Task 5's data lifts it to 7.5 for teams of 7–10 (see Task 4)
Fixes 1–6 built and running in the pilot	8	Hardy works and is trusted; its value is still shown team by team

Stage	Score	What has to be true
Pilot evidence	8.5–9	Pilot teams hit real conflicts every week; owners agree with Hardy at least 8 times in 10 (a target); at least one team turns on an automatic rule
Market pull	9–9.5	Teams that mix agent vendors choose Aurora because of Hardy, and audits turn into installs
10	Out of reach	The parts stay public, and GitHub can ship similar features

Building the fixes gets you to 8. Everything above 8 comes from evidence that teams want it, which only the pilot and the market can supply.

Updated framework: Hardy v2

Hardy v2: proof and any-vendor intake join the referee

Authority	GitHub sign-in, CODEOWNERS lanes, agent scopes signed by each owner Owners decide for their areas; the admin decides where no owner is listed
Intake	Claims from Aurora's agents and from any vendor's branches and pull requests A GitHub App, plus an MCP claim tool any agent can call
Prevent	Evening plan check: one owner per file, producers before consumers New: claims on functions, routes and tables; one owner a night for hot files
Detect	Lane watch at night; the merge queue tests branches together before morning New: notices to agents whose code relies on a contract that changed
Settle	Recipes for mechanical files; evidence and a blind pick for the rest Distinguishing tests run in Hardy's sandbox, out of the agents' reach
Decide	Conflict card to the owners; approval only in GitHub Slack and email for alerts; WhatsApp and iMessage for urgent ones
Learn	Every ruling saved as a precedent New: a team rulebook in the repo; automatic rules by opt-in, revocable
Prove	Collision audit of the last 90 days, then two weeks of shadow mode Signed log, weekly report and a published owner-agreement rate

Shaded rows are new layers in v2; a line starting with New is an upgrade to an existing layer

Hardy v2 · 8 layers, 2 new

Hardy v2 keeps the night flow from Task 1, adds two layers (Intake and Prove) and upgrades four others. Build the collision audit first: it needs no agents, works on any team's history today, and gives the pilot its baseline.

Milestone	Adds	Done when
H0 Collision audit	Replay of 90 days of history: overlaps, would-be conflicts, hot files	A team runs it on its own repo without sending code anywhere
H1 Plan check, any vendor	GitHub App intake, MCP claim tool, function and contract claims	Overlapping plans from two vendors' agents become one plan plus a ticket
H2 Shadow mode	Hardy reports what it would have done and changes nothing	Two weeks of reports for each pilot team
H3 Night watch and notices	Lane enforcement, notices to agents that rely on a changed contract	A changed contract warns the agent that uses it
H4 Evidence and decisions	Task 1's H3 and H4: evidence pack, conflict card, approval in GitHub	Owners decide planted conflicts in GitHub
H5 Rulebook and report	Precedents become rules; weekly report; signed log	A team edits its rulebook and reads its first weekly report

Task 3 — The standalone edge: can 5 go higher, and where to start

Yes, but not through technique. The 5 is low because every part Hardy uses is published. A standalone referee can only be defended by its position: who it stays neutral between, which standard it sets, which data only it holds, and where it is already installed. My estimate is that this can reach about 7 within 12 to 18 months. Going higher would need a network effect this market may not have.

Six sources of edge

Edge	Why it can hold	Who could take it	What to do
Own the measurement	Whoever publishes the numbers gets quoted, and every audit someone runs is a lead	Anyone with git skills; I found nobody publishing team-level collision numbers yet	Open-source the collision audit, publish a report on public repos, repeat it every quarter
Set the standard	If agents from many vendors declare their work the same way, the reference referee becomes the default	GitHub, or an agent vendor's own format	Publish a short open spec for claims and rulings, and propose it as an MCP extension
Precedents across teams	Anonymised rulings show which conflicts owners always approve, so new teams start with better default rules	Whoever has the most installs	Opt-in sharing of patterns only, never code
A trust record	A published owner-agreement rate and an open, audited rulebook take time to earn and can't be bought	Slow for anyone to copy	Publish the rate monthly; get an outside security review once teams pay
Installed where agents already are	Teams adopt what appears inside tools they already use	Platforms can bundle their own	GitHub Marketplace, Paperclip, and the MCP and plugin directories coding agents read from
Scope beyond code	Conflicts over migrations, infrastructure, feature flags and API contracts reach production, and code-only tools miss them	Specialist tools in each area	Add these resource types one at a time, after code works

How to widen the scope

Step	Scope	Why in this order
1	Code in one GitHub repo, teams of 5–10, Aurora's agents	Task 5 found real overlap at this size, and the pilot can measure it
2	Any vendor's agents on the same repo	Different vendors' agents conflicted twice as often in Xu et al.
3	Resources beside code: database migrations, infrastructure files, feature flags, environment settings	A collision here breaks production, not just a build
4	Contracts across repos, such as an API changed in one service and used in another	Producers-first ordering extends to this directly
5	Teams of 10–50	Overlap rose with team size in Task 5
6	GitLab and Bitbucket	Where GitHub's own features don't reach
7	Beyond code: agents writing to shared tickets, customer records or outbound email	The same problem in a much bigger market, with the least evidence today

Where to start: the first 90 days

1. **Weeks 1–2.** Package the Task 5 scripts as an open-source collision audit, a command-line tool and a GitHub Action that run on the team's own machine. Run it on 100 public repos.
2. **Weeks 3–4.** Publish the results and the method as an agent collision report, and invite readers to run the audit on their private repos.
3. **Weeks 3–6.** Write version 0.1 of a claims-and-rulings spec with a reference MCP server. Then raise it with MCP's working groups: an MCP proposal needs prior group discussion, a maintainer who agrees to sponsor it (without one within six months it goes dormant), and a reference implementation before it is final (MCP SEP guidelines).
4. **Weeks 5–8.** Ship Hardy's plan check as a GitHub App that accepts any vendor's agents. Recruit five design partners from audit users: teams of 5–10 running at least two agent vendors.
5. **Weeks 9–12.** Run shadow mode with those partners, then publish collisions caught and the owner-agreement rate.

6. **Throughout.** Contribute a claims plugin to Paperclip and join its open discussion on agent trust levels (RFC #379).

Steps 1 and 2 need one engineer for about two weeks, and the measurement scripts from Task 5 are a working start. Steps 3 to 5 need the two engineers experienced in agent or distributed systems that the earlier verdict set as a condition.

What each step is worth

Done	Standalone score (my estimate)
Today	5
Audit and report published, and cited by others	5.5
Spec adopted by at least one other tool or agent vendor	6–6.5
20 or more teams on Hardy across three or more vendors, agreement rate published	6.5–7
Cross-team precedents measurably improve a new team's first month	About 7

The ceiling stays near 7 because GitHub can ship a native version for its own users. The answer to that is breadth outside GitHub's own platform: other code hosts, resources beyond code, and a neutral standard that GitHub's competitors and agent vendors can adopt.

Task 4 — Is anyone already building this?

The pieces exist; the whole doesn't, as far as a public search can show. I looked across arXiv, GitHub, PyPI, Product Hunt, YC's directory, vendor blogs and tech news and found about 25 products, open-source projects and papers working on agents that collide in one codebase. None combines all six things Hardy does. Nobody I found lets each agent act for one person, settles their conflicts with evidence, sends the decision to the people who own the code, and learns from the rulings.

That isn't proof that nobody is building it privately; absence can't be proven. It is what is public as of 22 October 2026.

Who is closest

The six things Hardy does: **Owners** (each agent acts for one person; owners decide), **Prevent** (plans checked before work starts), **Settle** (evidence, not a vote), **Route** (decisions go to owners and the admin), **Learn** (rulings become rules), **Any vendor**. ● has it, ◐ partly, – no.

Who (stage)	Owners	Prevent	Settle	Route	Learn	Any vendor	Of 6
Nerveplane (research system, open source)	◐	●	–	◐	◐	●	3.5
Pawsly by Jindo AI (startup, seeking design partners)	–	●	◐	◐	–	●	3
GitHub Agent HQ (shipping)	◐	–	◐	◐	–	●	2.5
Shepherd (open source)	◐	●	–	–	–	●	2.5
GitKraken Team View and Kepler (shipping)	◐	◐	◐	–	–	●	2.5
ai-crew-sync (open source)	◐	◐	–	–	◐	●	2.5
CoordinationHub (open source)	◐	●	–	–	–	●	2.5
Claim Plane (research prototype)	◐	●	–	–	–	◐	2
Paperclip (the platform Aurora runs on)	◐	–	–	◐	–	◐	1.5
Hardy (planned)	●	●	●	●	●	●	6

What the closest three are, and how close:

- **Nerveplane** plans parallel agents before they start, so their scopes don't overlap and producers go before consumers. Deterministically it cut merge conflicts from 13 to 0. It is a research system by one author, with small live samples and scopes seeded by hand, and it doesn't address agents owned by different people.
- **Pawsly** drafts roles and dependencies for a team's agents, flags overlaps, checks agents against decisions taken in Slack, Linear and GitHub, and uses a separate verifier before human review. In the author's own CooperBench runs, matching code across four agents rose from 3 of 10 tasks to 9 of 10. The article invites design partners and doesn't say whether the agents belong to different people.
- **GitHub Agent HQ** runs agents from Anthropic, OpenAI, Google, Cognition and xAI in one place, with agent identity, policies, Slack and Linear integrations, and one-click conflict resolution. It has no shared plan check between agents and no owner-routed ruling between two agents' versions, but it is the player that could add them fastest. It is the biggest threat.

Others I checked cover one or two of the six: the agent-parley plugin (advisory file reservations), A2AMCP, Agent-MCP and SwarmCity (shared context and file coordination), Bernstein and Agent Orchestrator (overview: pre-merge checks; conflicts routed back to the session that caused them), Imbue Sculptor (one developer's parallel agents, with conflict help), CodeRabbit (resolves merge conflicts), Squad (one repo's Copilot agents with a shared decisions file), HumanLayer (routes approvals to people), Dock (agents that message each other and hand off work) and SkyDeck Agent Pods (persistent agent workspaces).

The field is moving fast. At least eight research papers on parallel agents colliding appeared in 2026 alone, including CooperBench, STALE, Claim Plane and NP-Bench.

Does this change the moat, the edge and the scope?

	Before	After the sweep and Task 5	Why
Moat inside Aurora	7	7, or 7.5 for teams of 7–10	The gap is real, and Task 5 found frequent collisions at 7–10 people once agents add volume
Standalone moat	5	5	Nobody combines the six, but several teams build each part
Edge	The combination	The combination, now sharper	Everyone is weak on owners, routing and learning; locks and worktrees are already common
Scope	Teams of 2–10	Teams of 5–10 that mix agent vendors first	Teams of 2–3 would see about one conflict a month

Should you go ahead?

Yes, with conditions. More harnesses help rather than hurt this idea: each vendor can only referee its own agents, and conflicts happen between them. Different vendors' agents conflicted 41.7% of the time against 19.8% for the same vendor (Xu et al.). The caveat is timing: in that data, pairs of different vendors' agents were only 0.5% of agents working at the same time, so mixed-vendor teams are still early. Having one or two others on the problem is normal at this stage; what matters is that nobody holds the owner-and-decision layer yet.

Go if you keep these five conditions:

1. Build Hardy so any vendor's agents can use it, with Aurora as its first home and the same open-source core underneath both.
2. Ship the collision audit within weeks, because the window is open now and many teams are building the pieces.
3. Compete on owners, routed decisions, evidence and learning, not on locks and worktrees, which are already common.
4. Sell first to teams of 5–10 that mix agent vendors, and treat Hardy as insurance for teams of 2–3.
5. Stop or pivot if GitHub ships shared plan checks with owner-routed rulings, or if pilot teams don't hit at least one conflict a week.

Task 5 — How often do small teams' agents collide?

It depends on team size. If each person's agent does about three days of that person's work a night, teams of 7–10 would hit about 1.5 to 4 textual conflicts a week, teams of 5–6 a median of about one every two weeks (one team in five far more), and teams of 2–3 about one a month. So the referee is a headline for teams of 7–10 and insurance for teams of 2–3.

What published studies show

Study	What it measured	Finding
Owhadi-Kareshk et al., 2019	267,657 merges by people in 744 repos	8.12% had textual conflicts
AgenticFlict, 2026	107,026 agent pull requests replayed against their base	27.67% conflicted, from about 15% to 32% depending on the agent
Xu et al., 2026	33,596 agent pull requests in 2,807 repos	40.2% of repos had agent pull requests open at the same time (53.4% within a week); 19.8% of those pairs conflicted with the same agent, 41.7% with different agents
Campos Junior and Murta, 2026	210,902 agent pull requests	29.5% of merges inside agent pull requests conflicted, and people resolved 96.1% of them

Study	What it measured	Finding
CooperBench, 2026	652 tasks shared by two agents	Working together, agents succeeded about 30% less often than alone; overlapping work caused 33.2% of failures
STALE, 2026	Clean merges that still break behaviour	1 in 834 runs on real merged Django pairs, but 97% of constructed interface changes
NP-Bench, 2026	Planning parallel agents before they start	Merge conflicts fell from 13 to 0 in deterministic scenarios

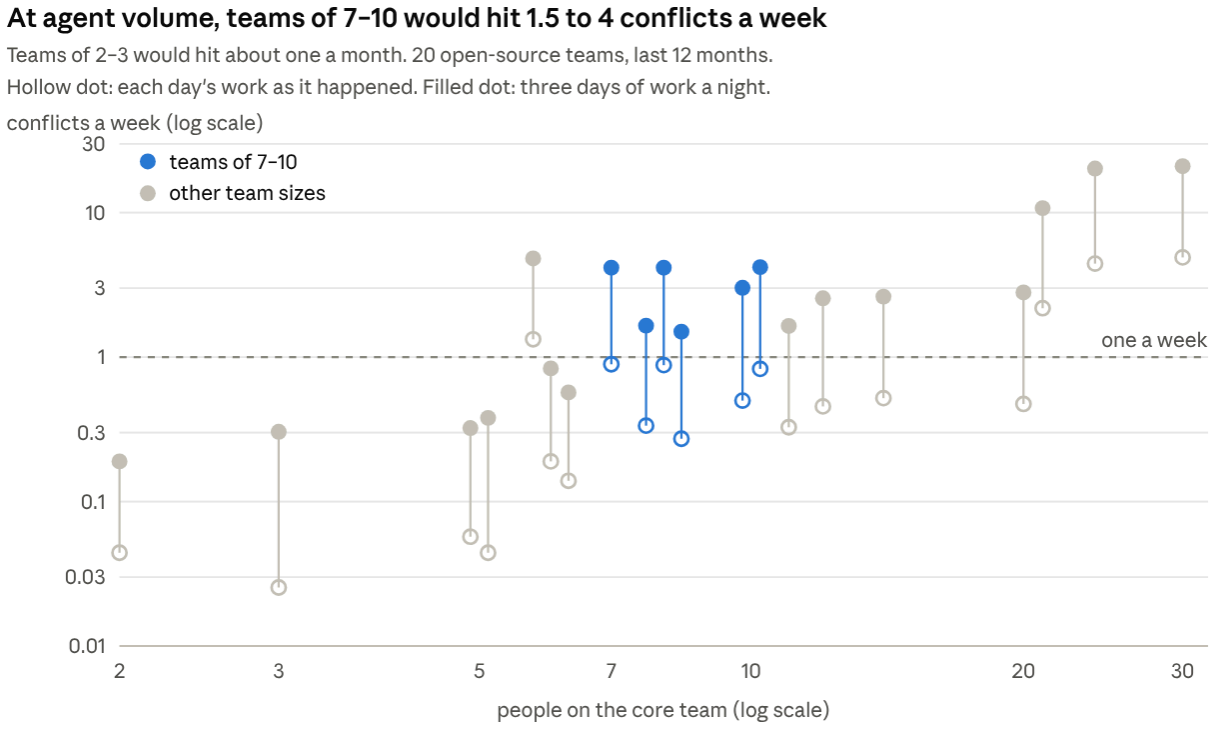
Agent pull requests conflict about three times as often as merges by people, and people clean up nearly all of it. None of these studies looks at teams of 2–10 people who each run their own agent, so I measured that.

What I measured

- **Teams.** 20 open-source product companies that build in public, over the last 12 months of git history. The core team is everyone with 10 or more commits; bots are excluded. 13 teams had 2–10 core people and 7 had 11–30.
- **Overlap.** Two people changing the same code file on the same day. Lockfiles, changelogs, translations, generated files and docs don't count.
- **Conflict.** I replayed 985 of these same-day pairs with git's own merge, as if both people had started from the same code that morning. I kept the 439 clean replays, where no other commit touched those files in between, and then only pairs from separate pull requests that weren't follow-ups.
- **Agent volume.** Three days of a person's work treated as one night of their agent's work.

When two people's same-day work touched the same file, replaying it in parallel produced a textual conflict 33% of the time for teams of 2–10 (34 of 103 pairs; 95% range 25–43%) and 24% for teams of 11–30 (20 of 82). Pairs that really were built on separate branches at the same time conflicted as often or more (10 of 19 and 14 of 50). I read 25 of the conflicting pairs: 21 were clearly separate tasks, one was follow-up work and three were unclear, so the true rate may be up to a tenth lower.

Core team	Teams	Working days that overlap a teammate	Would-be conflicts a week, at the pace it happened	At three days of work a night
2–3 people	2	5–7%	under 0.05	0.2–0.3 (about one a month)
5–6 people	5	6.5–38% (median 15%)	0.04–1.3 (median 0.14)	0.3–4.8 (median 0.6)
7–10 people	6	16–30% (median 23%)	0.3–0.9 (median 0.7)	1.5–4.2 (median 3.6)
11–30 people	7	21–47% (median 28%)	0.3–4.9 (median 0.5)	1.6–21 (median 2.8)



git history of 20 open-source teams, 22 Oct 2025 to 22 Oct 2026 · would-be conflicts = same-day file overlaps × measured conflict rate (33% for teams of 2–10, 24% for 11–30)

Tripling the volume multiplied overlaps about five times, because each person touches more files and so meets teammates more often. That is why agents turn a rare event for teams of 7–10 into a weekly one.

Insurance or headline?

- **Teams of 7–10: headline.** At agent volume they would hit about 1.5 to 4 conflicts a week, mostly in source code that someone has to untangle by hand before the morning's work can merge.
- **Teams of 5–6: it depends on the team.** The median is about one every two weeks, but one team in five would hit nearly five a week. The collision audit shows which kind a team is before it buys.
- **Teams of 2–3: insurance.** About one a month even at agent volume. Lead with the overnight work itself and keep Hardy as the safety net.

What these numbers can't tell you

- **They come from people, not agents.** Agents don't overhear each other the way teammates do, which would push real rates up. Dependent work that my filters missed pushes my rates down slightly.
- **The three-day volume is an assumption.** Agent output per night will vary widely between teams.
- **They count textual conflicts only.** Clean merges that break behaviour were rare in real reviewed history but common when agents change interfaces blind (STALE, above).
- **Open-source teams may differ from private ones,** and four of these teams already credit AI assistants in a third or more of their commits (Formbricks, Langfuse, Novu, Electric).
- **20 teams is a small sample.** Single-team figures are noisy; the rise with team size is the solid part.

The pilot can settle it: run the collision audit on each pilot team's repo before starting, then count real conflicts during the pilot. If teams of 5–10 see at least one a week, Hardy leads the pitch; if not, it stays the safety net.

What to add to the pilot

The full Hardy build takes weeks, and the applications are due on 31 October and 2 November. Before then, add only what produces evidence; build the rest after.

Add	When	Claude Code does	Only you do	What it gives you
Collision audit for each tester team	From the first tester calls	Packages the Task 5 scripts into one command that runs inside a team's own copy of its repo and sends nothing anywhere	Ask each team to run it, or run it with them on the call	Each team's own collision number before its first night
Morning overlap check	M6, 23–26 Oct	Before the morning email, checks every pair of that night's agent branches with git's merge check and lists, in the email, the files both changed and whether they would conflict. It holds nothing and changes nothing.	Nothing	Real counts of agent collisions per night, the pilot's version of Task 5
Overlap numbers on the numbers page	M6	Adds overlaps per night, would-be conflicts and minutes to decide	Nothing	Figures for the applications
Tester mix	Now	Waits	Prefer teams of 5–10 where at least two people already use agents, ideally different ones such as Claude Code and Codex; ask on each call which files everyone touches	Teams where conflicts actually happen
Application wording	By 31 Oct	Waits	Use the evidence below	A claim backed by your own data

Add	When	Claude Code does	Only you do	What it gives you
Plan check with file claims	After 2 Nov (H1)	Each agent lists the files it plans to change; overlapping plans become one plan plus a ticket	Approve the change	The first step that prevents collisions instead of counting them

The applications can say three things, each backed by this doc: nobody public combines owner authority, evidence, routed decisions and learning (Task 4); on 12 months of public history, teams of 7–10 would hit about 1.5 to 4 conflicts a week at agent volume (Task 5); and the pilot now counts real overlaps every night. Call the Task 5 figure "would-be conflicts measured on history", not a measured agent rate.

Leave these until after the applications: Slack buttons, WhatsApp, iMessage, the blind judge, distinguishing tests and automatic rulings. Email stays the only decision channel, as Milestone 5 built it, with the inbox-equals-approval gap still recorded.

Decide after four weeks of pilot nights. If the teams of 5–10 see at least one conflict a week, Hardy leads the pitch. If they don't, lead with the overnight work and keep Hardy as the safety net.

Sources

Research and studies

- Owhadi-Kareshk et al., predicting merge conflicts (2019)
- AgenticFlict: merge conflicts in agent pull requests (2026)
- Xu, Subramanian and Karthik: agent pull requests and merge conflict rates (2026)
- Campos Junior and Murta: how AI coding agents resolve merge conflicts (SBES 2026)
- CooperBench (2026)
- STALE: passes alone, fails together (2026)
- NP-Bench and Nerveplane (2026)
- Claim Plane (2026)
- S*: test-time scaling for code generation
- CaMeL: defeating prompt injections by design, summary
- Authenticated delegation for AI agents (MIT Media Lab)
- OpenAI: practices for governing agentic AI systems
- Model judges favour their own outputs (NeurIPS 2024)

- METR: recent frontier models are reward hacking
- Debate with more persuasive models (2024)
- Multi-agent debate versus majority voting (ICML 2024)
- Agents growing more agreeable in debate (arXiv 2509.05396)
- Confidence elicitation in language models (arXiv 2306.13063)
- Inflated verbalised confidence (arXiv 2604.01457)
- Confidence of tool-using agents (arXiv 2601.07264)
- Semantic entropy, journal-club summary
- Collaborative Gym
- Magentic-UI (Microsoft Research)
- MAST: why multi-agent systems fail
- Agents of Chaos, Radware write-up
- Agents reporting tasks as complete (arXiv 2602.20021)
- Anthropic: how we built our multi-agent research system
- Cognition on multi-agent work, second post
- METR's 2025 developer trial (eWeek), no control group in 2026 (I Programmer), what the productivity data shows (Level Up)

Products and projects

- Pawsly by Jindo AI (HackerNoon)
- GitHub Agent HQ (SD Times)
- Shepherd (Product Hunt)
- GitKraken Team View, GitKraken Kepler and conflict tools
- ai-crew-sync
- CoordinationHub
- Paperclip release v2026.916.0, Paperclip RFC #379 on agent trust levels
- agent-parley plugin
- A2AMCP, Agent-MCP, SwarmCity
- Open-source agent orchestrators, including Bernstein and Agent Orchestrator (Augment Code)
- Imbue Sculptor (DevTune)
- CodeRabbit merge-conflict resolution
- Squad (GitHub blog)
- HumanLayer
- Dock (YC)

- SkyDeck Agent Pods
- Claude Code agent teams
- Claude Tag identity and access model
- Cursor parallel agents (Medium), Cursor 3 agents window, Cursor forum report
- MeetGeek, CoAgentor, AgentCall

Platform and protocol documentation

- Slack: verifying requests
- Vonage: WhatsApp provisioning
- Zendesk: Apple Messages for Business
- Photon iMessage adapter
- GitHub merge queue (Paradime guide), Mergify parallel scopes
- MCP SEP guidelines

Own measurement

Git history from 22 October 2025 to 22 October 2026 of 20 public repositories: documenso/documenso, formbricks/formbricks, plausible/analytics, unkeyed/unkey, hatchet-dev/hatchet, openstatusHQ/openstatus, dubinc/dub, mfts/papermark, excalidraw/excalidraw, hoppscotch/hoppscotch, umami-software/umami, chatwoot/chatwoot, novuhq/novu, triggerdotdev/trigger.dev, electric-sql/electric, tldraw/tldraw, inngest/inngest, langfuse/langfuse, twentyhq/twenty and Infisical/infisical. The analysis scripts were shared alongside this doc.